

Librairies d'effets A-Frame : effects-model.js / effects-vertices.js

Ces deux librairies fournissent des composants A-Frame déclenchés par des **events** (via `debug-keyboard` ou tout autre émetteur d'événement sur l'entité, dont `scheduler`). Elles s'appliquent à une entité portant un modèle 3D (`obj-model` ou `gltf-model`).

- **effects-model.js** : effets qui agissent directement sur le(s) matériau(x) du modèle 3D.
- **effects-vertices.js** : effets qui agissent sur des sphères (ou sprites) créées à la position de chaque vertex du fichier source, lu comme un fichier texte.

Prérequis généraux

- La page doit être servie en **http(s)** (Live Server, etc.) — pas en `file://`, car certains effets font un `fetch()` du fichier `.obj` source.
- Les paramètres numériques comme `radius`, `amplitude`, `spread`, `distance`, `height` s'expriment dans les **unités locales du modèle** (celles du fichier `.obj`), pas en mètres absolus : si l'entité a un `scale` différent de `1 1 1`, ce facteur s'applique aussi à ces valeurs.
- Un composant `vertices-xxx` (sauf `vertices` lui-même) nécessite que l'événement `vertices` ait déjà été déclenché au moins une fois sur la même entité, pour que les sphères existent.

Convention de rejeu

Ce qui se passe si on redéclenche le même événement varie désormais selon l'effet :

- **Réversible** : toggle classique, le 2^e appel annule/inverse le 1^{er}.
- **Idempotent** : chaque appel repart du même point de référence, résultat équivalent à chaque fois.
- **Cumulatif** : chaque appel continue depuis l'état laissé par le précédent, sans retour en arrière possible.
- **Destructif** : consomme une ressource finie ; une fois épuisée, un rejeu n'a plus d'effet.

La plupart des effets ont quitté le toggle réversible pour aller vers cumulatif ou destructif — cohérent avec l'idée qu'une vanité ne revient pas en arrière. `vertices` reste volontairement réversible, en tant que couche de base dont dépendent les autres. Quelques effets restent **paramétrables** : selon la valeur de `destroy`, ils basculent d'une catégorie à l'autre.

effects-model.js

Composants agissant sur le modèle 3D lui-même (son ou ses matériaux `three.js`).

model-wireframe — cumulatif

Objet : bascule en mode filaire (`wireframe`) les matériaux du modèle, un par un. **Événement déclencheur** : `wireframe`. Chaque déclenchement supplémentaire fait passer **un nouveau matériau** (parmi ceux pas encore touchés) en `wireframe`. Une fois tous les matériaux convertis, l'effet est épuisé (avertissement en console).

Paramètre	Type	Défaut	Description
<code>color</code>	couleur (<i>vide</i>)		Si renseigné, force cette couleur pour chaque matériau au moment où il passe en <code>wireframe</code> .

Syntaxe :

```
model-wireframe
model-wireframe="color: #00ff00"
```

Notes : en mode wireframe, three.js n'affiche jamais la texture (map) du matériau, seule sa couleur compte — c'est un comportement normal du moteur, pas une limite du composant. L'effet n'ayant plus de retour en arrière, la couleur d'origine n'est plus sauvegardée/restaurée.

model-transparent — cumulatif

Objet : réduit l'opacité du modèle par paliers, depuis sa valeur **actuelle**, jusqu'à un plancher. **Event déclencheur** : `transparent`. Chaque déclenchement retranche `step` à l'opacité en cours, sans jamais redescendre en dessous de `min`, et sans retour à l'opacité pleine.

Paramètre	Type	Défaut	Description
<code>step</code>	nombre (0-1)	0.2	Réduction d'opacité appliquée à chaque déclenchement.
<code>min</code>	nombre (0-1)	0.05	Opacité plancher, jamais franchie.

Syntaxe :

```
model-transparent="step: 0.2; min: 0.05"
```

Notes : bascule instantanée, sans animation (contrairement à `model-desappear`).

model-desappear — idempotent

Objet : anime une baisse progressive de l'opacité du modèle jusqu'à une valeur donnée, puis revient automatiquement à l'opacité pleine — le tout en un seul aller-retour autonome. **Event déclencheur** : `desappear`. Chaque déclenchement rejoue l'aller-retour complet, indépendamment des précédents.

Paramètre	Type	Défaut	Description
<code>opacity</code>	nombre (0-1)	0.3	Opacité cible de la disparition.
<code>duration</code>	nombre (ms)	2000	Durée de chaque trajet (aller, puis retour).

Syntaxe :

```
model-desappear="opacity: 0.1; duration: 3000"
```

Notes : ne nécessite plus un second event pour revenir — un seul déclenchement suffit à jouer le cycle complet.

model-bluescreen — destructif

Objet : évoque une panne matérielle soudaine — flash bref d'une couleur d'alerte, puis extinction brutale et instantanée (sans transition), en contraste volontaire avec `model-desappear`. **Event déclencheur** : `bluescreen`. Le crash est **définitif** : un déclenchement supplémentaire une fois la panne survenue n'a plus d'effet (avertissement en console).

Paramètre	Type	Défaut	Description
<code>color</code>	couleur	#0000aa	Couleur du flash d'alerte (bleu façon écran bleu par défaut).
<code>duration</code>	nombre (ms)	400	Durée du flash avant l'extinction brutale.

Syntaxe :

```
model-bluescreen="color: #0000aa; duration: 400"
```

model-reboot — cumulatif puis destructif

Objet : clignotement on/off de toute l'entité, comme un vieil écran qui agonise, un peu plus dégradé (fréquence plus basse, clignotement plus long) à chaque nouvelle tentative. **Event déclencheur** : `reboot`. Après `attempts` tentatives, le redémarrage échoue **définitivement** (extinction permanente). Avec `attempts` : 1, la toute première tentative échoue déjà.

Paramètre	Type	Défaut	Description
<code>duration</code>	nombre (ms)	2000	Durée du clignotement de la 1 ^{re} tentative.
<code>frequency</code>	nombre (clignotements/s)	8	Vitesse du clignotement au départ de la 1 ^{re} tentative.
<code>slowdown</code>	booléen	true	Si vrai, le clignotement ralentit progressivement vers la fin de chaque tentative.
<code>attempts</code>	nombre	3	Nombre de tentatives avant panne définitive (0 = jamais de panne).

Syntaxe :

```
model-reboot="duration: 2000; frequency: 8; slowdown: true; attempts: 3"
```

model-delete — destructif

Objet : supprime définitivement l'entité (modèle et tout ce qui lui est rattaché) — géométrie et matériaux libérés (`dispose`), visibilité mise à `false`.

Event déclencheur : `delete`. Un déclenchement supplémentaire une fois supprimé n'a plus d'effet (avertissement en console).

Pas de paramètre.

Syntaxe :

```
model-delete
```

effects-vertices.js

Composants agissant sur des sphères (ou sprites) créées à partir des positions des vertices, lues directement dans le fichier `.obj` source.

vertices — réversible

Objet : composant central de la librairie. Rend le modèle transparent, lit le fichier `.obj` comme du texte, en extrait les positions de vertices, et crée une sphère à chaque position. **Event déclencheur** : `vertices` (toggle : un second déclenchement retire les sphères et restaure l'opacité d'origine du modèle).

Paramètre	Type	Défaut	Description
<code>radius</code>	nombre	0.001	Rayon des sphères.
<code>color</code>	couleur	#ffffff	Couleur des sphères.
<code>opacity</code>	nombre (0-1)	0.15	Opacité du modèle pendant que les sphères sont affichées.
<code>ratio</code>	nombre (0-1)	1	Proportion des vertices utilisée (échantillonnage régulier, pas aléatoire). Utile si le modèle a beaucoup de points ou si plusieurs modèles utilisent ce composant.

Syntaxe :

```
vertices="radius: 0.02; color: white; opacity: 0.15; ratio: 0.1"
```

Notes : le fichier `.obj` n'est lu/parsé qu'une seule fois (mis en cache) ; les toggles suivants réutilisent les positions déjà extraites. Composant volontairement resté réversible : c'est la couche de base dont dépendent tous les composants `vertices-xxx` ci-dessous, qui eux ne le sont plus.

vertices-desappear — cumulatif

Objet : anime l'opacité des sphères de vertices (pas celle du modèle) par paliers, depuis sa valeur actuelle, jusqu'à un plancher. **Event déclencheur** : `vertices-desappear`. Chaque déclenchement retranche `step` à l'opacité en cours, sans retour en arrière.

Paramètre	Type	Défaut	Description
<code>step</code>	nombre (0-1)	0.2	Réduction d'opacité appliquée à chaque déclenchement.
<code>min</code>	nombre (0-1)	0	Opacité plancher, jamais franchie.
<code>duration</code>	nombre (ms)	2000	Durée de la transition.

Syntaxe :

```
vertices-desappear="step: 0.2; min: 0; duration: 3000"
```

Notes : toutes les sphères partagent un seul matériau (pour la performance), donc l'opacité s'applique à l'ensemble du nuage de points, pas sphère par sphère.

vertices-falling — idempotent ou destructif (selon `destroy`)

Objet : chaque sphère descend (en position locale `y`) jusqu'à `y=0`, comme des flocons qui tombent au sol. **Event déclencheur** : `vertices-falling`.

- `destroy: false` (défaut) → **idempotent** : les sphères restent au sol ; un rejeu les "anime" de 0 à 0, sans changement visible.
- `destroy: true` → **destructif** : les sphères sont supprimées une fois au sol, effet épuisé après le premier appel.

Paramètre	Type	Défaut	Description
<code>duration</code>	nombre (ms)	2000	Durée de la chute.
<code>destroy</code>	booléen	false	Si vrai, la sphère est supprimée une fois arrivée au sol (comme un flocon qui fond).

Syntaxe :

```
vertices-falling="duration: 3000; destroy: true"
```

vertices-glitch — idempotent

Objet : perturbation aléatoire et répétée de la position de chaque sphère pendant une courte durée, comme un artefact d'affichage avant une panne, puis retour exact à la position d'origine. **Event déclencheur** : `vertices-glitch`. Rejouable à volonté sans dérive (chaque sphère revient précisément à sa position sauvegardée avant le glitch) ; le motif aléatoire diffère à chaque appel mais reste statistiquement équivalent.

Paramètre	Type	Défaut	Description
amplitude	nombre	0.02	Distance maximale de déplacement aléatoire à chaque saut.
duration	nombre (ms)	800	Durée totale du glitch.
frequency	nombre (sauts/s)	60	Nombre de sauts aléatoires par seconde.

Syntaxe :

```
vertices-glitch="amplitude: 0.02; duration: 800; frequency: 60"
```

vertices-corrupt — cumulatif

Objet : déplace instantanément (sans animation) un pourcentage de sphères, dans une direction arbitraire — évoque un bit qui bascule (bit-rot), sans notion de gravité contrairement à `vertices-falling`. **Event déclencheur** : `vertices-corrupt`. Chaque déclenchement corrompt un **nouveau lot** de sphères, choisi parmi celles pas encore touchées, sans jamais restaurer les précédentes. Une fois toutes les sphères corrompues, l'effet est épuisé (avertissement en console).

Paramètre	Type	Défaut	Description
percentage	nombre (0-1)	0.1	Proportion du total des sphères corrompue à chaque déclenchement.
amplitude	nombre	0.05	Distance maximale du déplacement aléatoire.

Syntaxe :

```
vertices-corrupt="percentage: 0.1; amplitude: 0.05"
```

vertices-scatter — cumulatif ou destructif (selon destroy)

Objet : chaque sphère monte et se disperse horizontalement de façon aléatoire, comme de la cendre qui se dissout — dissolution plutôt que chute (mouvement inverse de `vertices-falling`). **Event déclencheur** : `vertices-scatter`.

- `destroy: true` (défaut) → **destructif** : les sphères sont supprimées une fois dissoutes, effet épuisé après le premier appel.
- `destroy: false` → **cumulatif** : chaque rejeu disperse davantage, en partant de la position déjà dispersée par le précédent appel.

Paramètre	Type	Défaut	Description
duration	nombre (ms)	3000	Durée de la montée.
height	nombre	0.5	Distance de montée verticale (avec une variation aléatoire jusqu'à $\times 1.5$).
spread	nombre	0.3	Dispersion horizontale aléatoire maximale (x/z).
destroy	booléen	true	Si vrai, la sphère est supprimée une fois la dissolution terminée.

Syntaxe :

```
vertices-scatter="duration: 3000; height: 0.5; spread: 0.3; destroy: true"
```

vertices-binary-rain — idempotent, autonome

Objet : remplace visuellement l'idée des sphères statiques par une pluie continue de chiffres "0"/"1" façon Matrix, positionnés aux vertices et tombant en boucle. **Event déclencheur** : `vertices-binary-rain`. Démarre l'animation, qui s'arrête désormais **d'elle-même** après `lifetime` ms (chaque relance tire un nouveau motif 0/1, statistiquement équivalent). Un second déclenchement pendant que ça tombe reste possible comme

raccourci d'arrêt manuel anticipé.

Paramètre	Type	Défaut	Description
ratio	nombre (0-1)	0.1	Proportion des vertices utilisée.
size	nombre	0.05	Taille des sprites 0/1.
color	couleur	#00ff00	Couleur des chiffres.
distance	nombre	0.3	Hauteur de chute avant que chaque chiffre reboucle en haut.
duration	nombre (ms)	2000	Temps pour parcourir <i>distance</i> (détermine la vitesse de chute).
lifetime	nombre (ms)	4000	Durée totale avant arrêt automatique (0 = infini, comme avant).

Syntaxe :

```
vertices-binary-rain="ratio: 0.1; size: 0.05; color: #00ff00; distance: 0.3; duration: 2000; lifetime: 4000"
```

Notes : chaque chiffre démarre avec un décalage aléatoire pour éviter une chute synchronisée. Nécessite que `vertices` ait déjà été déclenché (pour avoir les positions en cache), même si les sphères elles-mêmes ne sont pas utilisées par cet effet.

vertices-erosion — cumulatif

Objet : un pourcentage des sphères **restantes** (pas des sphères originales) disparaît, chacune instantanément, mais à des instants étalés (et légèrement irréguliers) sur toute la durée — l'usure du temps plutôt qu'une commande unique. **Event déclencheur** : `vertices-erosion`. Chaque déclenchement continue l'effacement là où le précédent s'est arrêté ; finit par s'épuiser de lui-même une fois toutes les sphères effacées.

Paramètre	Type	Défaut	Description
duration	nombre (ms)	5000	Durée totale de l'érosion.
percentage	nombre (0-1)	1.0	Proportion des sphères restantes qui finissent par disparaître à ce déclenchement.
jitter	nombre	0.3	Irrégularité du rythme de disparition (0 = métronomique, plus élevé = plus erratique).

Syntaxe :

```
vertices-erosion="duration: 5000; percentage: 1.0; jitter: 0.3"
```

Tableau récapitulatif

Composant	Event	Fichier	Catégorie
model-wireframe	wireframe	effects-model.js	Cumulatif
model-transparent	transparent	effects-model.js	Cumulatif
model-desappear	desappear	effects-model.js	Idempotent
model-bluescreen	bluescreen	effects-model.js	Destructif
model-reboot	reboot	effects-model.js	Cumulatif → Destructif
model-delete	delete	effects-model.js	Destructif
vertices	vertices	effects-vertices.js	Réversible
vertices-desappear	vertices-desappear	effects-vertices.js	Cumulatif
vertices-falling	vertices-falling	effects-vertices.js	Idempotent ou destructif (<code>destroy</code>)
vertices-glitch	vertices-glitch	effects-vertices.js	Idempotent
vertices-corrupt	vertices-corrupt	effects-vertices.js	Cumulatif
vertices-scatter	vertices-scatter	effects-vertices.js	Cumulatif ou destructif (<code>destroy</code>)
vertices-binary-rain	vertices-binary-rain	effects-vertices.js	Idempotent, autonome
vertices-erosion	vertices-erosion	effects-vertices.js	Cumulatif

